Lab 4: Introduction to MPI Communication: Ping-Pong and Ring Communication
Course: Parallel Computing
Time Limit: 120 Minutes
Submission: End of lab

Learning Objectives

By completing this lab, you will be able to:

- Initialize and finalize an MPI program correctly
- Use blocking point-to-point communication:
    - MPI_Send
    - MPI_Recv
    - MPI_Sendrecv
- Implement a ping-pong communication pattern
- Implement ring (neighbour) communication
- Reason about deadlock and message matching
- Measure communication time using MPI_Wtime()

Allowed and Disallowed MPI Calls

Allowed communication routines

You may use **only**:

- MPI_Send
- MPI_Recv
- MPI_Sendrecv

Allowed utility routines

- MPI_Init
- MPI_Finalize
- MPI_Comm_rank
- MPI_Comm_size
- MPI_Wtime

Not allowed

- Collective communication (MPI_Bcast, MPI_Reduce, etc.)
- Non-blocking calls (MPI_Isend, MPI_Irecv)

- Custom MPI datatypes
- memcpy, struct

Program Requirements

Your program must:

1. Be written in **C**
2. Compile using mpicc (with compiler optimization)
3. Accept **one command-line argument (with core binding)**:
    - pingpong
    - ring
4. Behave differently based on the selected mode
5. Finalize MPI cleanly before exiting

Build and Run Instructions (Required)

Compilation

mpicc -O3 lab4.c -o lab4

Ping-pong execution

mpirun -np 2 lab4 pingpong

Ring execution

mpirun -np P lab4 ring

where $P \geq 2$.

Message Representation

Message size control

Messages must be sent as **raw byte buffers**:

char *buffer = malloc(msg_size);

- The **message size (in bytes)** is exactly the size of the buffer.
- Message content is irrelevant.
- Message size is controlled **only** by msg_size.

You must **not** use:

- int arrays
- struct

- embedding values inside the buffer

Correctness Rule (No Payload Data)

Correctness must be verified using **MPI message tags**, not message content.

- Each ping-pong iteration has an iteration number k
- The message tag must be derived from k
- Use modulo arithmetic to keep tags in range

Example rule (you must implement this logic yourself):

tag = k % TAG_RANGE

The reply message must use the **same tag**.

Part A – Ping-Pong Communication

Objective

Implement a ping-pong exchange between **exactly two MPI processes**.

Requirements

- Run only with -np 2
- Perform K = 10,000 ping-pong iterations
- Use at least **5 message sizes**, e.g.:

1, 8, 64, 512, 4096 bytes

- Use blocking communication
- Avoid deadlock
- Measure **round-trip time (RTT)** on rank 0 using MPI_Wtime()

Timing

- Measure total time for K iterations
- Compute average RTT:

avg_RTT = total_time / K

- Estimate one-way latency as:

one_way ≈ avg_RTT / 2

(this is an approximation)

Part B – Ring Communication

Objective

Implement neighbour communication in a ring topology.

Ring definition

For P processes with ranks 0 … P-1:

- Right neighbour:

(rank + 1) % P

- Left neighbour:

(rank - 1 + P) % P

Requirements

- Run with P ≥ 2
- Each rank:
    - sends its rank to the right neighbour
    - receives a value from the left neighbour
- Use **only** MPI_Sendrecv
- Print one line per rank confirming the exchange

Example output format (order may vary):

Rank r: sent r to right, received x from left

Deadlock Awareness (Conceptual)

You are expected to reason about:

- Why naïve MPI_Send → MPI_Recv patterns can deadlock
- Why MPI_Sendrecv avoids deadlock in a ring

You do not need to submit code demonstrating deadlock; only correct logic is required.

Deliverables

Submit:

1. **Source code** (lab4.c)
2. **Short report** (1–2 pages or text file) including:
    - Message sizes tested
    - Average RTT results
    - Estimated one-way latency
    - Brief explanation of:

- how message size is controlled
- how correctness is verified
- why your implementation avoids deadlock

By the end of this lab, you will be able to:

1. Compile and run MPI programs using mpirun
2. Identify MPI ranks and reason about process-to-process communication
3. Use blocking point-to-point communication (MPI_Send, MPI_Recv)
4. Safely exchange data between neighbours using MPI_Sendrecv
5. Measure communication time using MPI_Wtime
6. Explain common MPI deadlock scenarios and how to avoid them

Allowed MPI Functions (Communication)

You may **only** use the following MPI communication routines:

- MPI_Send
- MPI_Recv
- MPI_Sendrecv

You will also need standard MPI setup/utility calls:

- MPI_Init
- MPI_Comm_size
- MPI_Comm_rank
- MPI_Finalize
- MPI_Wtime

Do **not** use:

- MPI_Bcast, MPI_Reduce, MPI_Scatter, MPI_Gather
- Non-blocking calls (MPI_Isend, MPI_Irecv)
- Any collective communication

Part 0 – Setup and Environment

Compilation

mpicc -O3 --lab4.c -o lab4

Execution Examples

mpirun -- bind-to core -np 2 lab1 pingpong

mpirun -- bind-to core -np 8 lab1 ring

You may run all processes on a single machine.

Part A – Ping-Pong Communication

Goal of this experiment

The goal of the ping-pong experiment is to understand **how communication time changes as the message size changes**.

You will:

- send messages of **different sizes (in bytes)**,
- measure the **round-trip time (RTT)**, and
- observe how small and large messages behave differently.

A1. What controls the message size?

The message size is exactly the number of bytes you allocate and send.

This lab uses **byte buffers** to control message size precisely.

What this means

If you write:

```
char *buffer = malloc(msg_size);
MPI_Send(buffer, msg_size, MPI_BYTE, ...);
```

Then MPI sends:

- msg_size elements
- each of type MPI_BYTE (1 byte)

Examples

msg_size	Bytes sent
1	1 byte
8	8 bytes
64	64 bytes
512	512 bytes
4096	4096 bytes

Important clarifications

- The **content** of the buffer does not matter
- You do **not** need to initialize the buffer

- The buffer exists **only to control message size**

A2. Message Content and Correctness (Using Tags)

In this part:

- You will **not embed any values inside the message buffer**
- Correctness will be checked using **MPI message tags**

How correctness works

- Each ping-pong iteration has an iteration number k
- You derive a message tag from k
- The receiver checks the received tag

Because MPI tags have a limited range, use modulo arithmetic.

Required rule for tags

Let:

#define TAG_RANGE 1000

For iteration k:

int tag = k % TAG_RANGE;

Sender (rank 0)

MPI_Send(buffer, msg_size, MPI_BYTE, 1, tag, MPI_COMM_WORLD);

Receiver (rank 1)

```
MPI_Status status;
MPI_Recv(buffer, msg_size, MPI_BYTE, 0, MPI_ANY_TAG,
      MPI_COMM_WORLD, &status);

if (status.MPI_TAG != tag) {
   printf("Tag mismatch at iteration %d\n", k);
}
```

The reply message uses the **same tag**.

A3. Ping-Pong Communication Pattern

The ping-pong pattern is fixed:

- **Rank 0**: Send → Receive
- **Rank 1**: Receive → Send

This ordering prevents deadlock.

A4. Timing the Ping-Pong

What you measure

- Measure **round-trip time (RTT)** on **rank 0 only**
- Use MPI_Wtime()

What RTT means

RTT includes:

- time to send the message from rank 0 to rank 1
- time to send the reply back from rank 1 to rank 0

Timing structure (rank 0)

```
double t0 = MPI_Wtime();

/* ping-pong loop */

double t1 = MPI_Wtime();
double total_time = t1 - t0;
```

Average RTT

If you run K iterations:

Average RTT = total_time / K

Estimated one-way latency:

One-way ≈ RTT / 2 (approximation)

A5. Required Experiments

You must test:

Message sizes (at least 5)

1, 8, 64, 512, 4096 (bytes)

Iterations

- K = 10,000 (you may reduce K for very large messages)

A6. Required Output (Rank 0)

Produce a table similar to:

Message Size (bytes)	Iterations	Avg RTT
1	10000	...
8	10000	...
64	10000	...

512	10000	...
4096	10000	...

Part A – Ping-Pong Communication

Objective

Implement a classic **ping-pong communication** between **two MPI processes**.

- Rank 0 sends a message to rank 1
- Rank 1 receives the message and sends it back
- Rank 0 receives the reply
- This exchange is repeated multiple times to measure communication time

A1. Program Requirements

- Exactly **2 MPI processes** (-np 2)
- Use:
    - MPI_Send
    - MPI_Recv
- Use different message tags for each direction

A2. Message Content

Each message should include:

- An integer value (e.g., iteration number)
- Optionally a small array for larger message sizes

This allows you to verify correctness.

A3. Iterations and Message Sizes

Use:

- Number of iterations: **K = 10,000**
- Test at least **5 message sizes**, for example:

1, 8, 64, 512, 4096, 65536 bytes

(You may reduce K for very large messages.)

A4. Timing Methodology

- Measure time **only on rank 0**
- Use MPI_Wtime()
- Start timing **before** the ping-pong loop
- Stop timing **after** the loop

Compute:

- Average round-trip time (RTT):

RTT_avg = total_time / K

- Estimated one-way latency:

latency ≈ RTT_avg / 2

Clearly state that this is only an estimate.

A5. Required Output (Rank 0 Only)

Print a table similar to:

Message Size (bytes) | Iterations | Avg RTT (μs) | Est. One-way (μs)

A6. Short Questions (Answer in Text File)

1. Why is timing performed on rank 0 only?
2. Why is RTT/2 only an approximation of one-way latency?

Part B – Ring Communication

Objective

Implement **neighbour communication in a ring topology** using MPI_Sendrecv.

B1. Ring Topology Definition

For **P processes** with ranks 0 … P-1:

- Right neighbour:

right = (rank + 1) % P

- Left neighbour:

left = (rank - 1 + P) % P

B2. One-Step Ring Exchange

Each process:

1. Sends a value to its **right** neighbour
2. Receives a value from its **left** neighbour

Initial value:

send_value = rank;

After the exchange, each rank should receive:

(rank - 1 + P) % P

B3. Deadlock Discussion (Conceptual)

Consider this code pattern:

MPI_Send(... right ...);

MPI_Recv(... left ...);

Question:
Why can this deadlock if all processes execute it simultaneously?

You do **not** need to implement this version. Just understand the issue.

B4. Required Implementation: MPI_Sendrecv

Implement the ring exchange using:

```
MPI_Sendrecv(&send_value, 1, MPI_INT, right, TAG,
        &recv_value, 1, MPI_INT, left,  TAG,
        MPI_COMM_WORLD, &status);
```

This avoids deadlock while using blocking communication.

B5. Output

Each process should print:

Rank r: sent X to rank right, received Y from rank left

(Interleaved output is acceptable.)

Starter Code (Provided)

```
#include <mpi.h>
#include <stdio.h>
#include <string.h>
```

```
#define TAG01 100
#define TAG10 101
#define TAGR  200

void run_pingpong(int rank);
void run_ring(int rank, int size);

int main(int argc, char** argv) {
  MPI_Init(&argc, &argv);

  int rank, size;
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  MPI_Comm_size(MPI_COMM_WORLD, &size);

  if (argc < 2) {
    if (rank == 0)
      printf("Usage: %s pingpong|ring\n", argv[0]);
    MPI_Finalize();
    return 0;
  }

  if (strcmp(argv[1], "pingpong") == 0) {
    if (size != 2) {
      if (rank == 0) printf("Ping-pong requires -np 2\n");
    } else {
      run_pingpong(rank);
    }
  }
  else if (strcmp(argv[1], "ring") == 0) {
```

```
    if (size < 2) {
      if (rank == 0) printf("Ring requires -np >= 2\n");
    } else {
      run_ring(rank, size);
    }
  }

  MPI_Finalize();
  return 0;
}
```

Deliverables

Submit:

1. Lab4.c
    - Contains both ping-pong and ring implementations
2. answers.txt
    - Ping-pong timing table
    - Short answers to questions in Part A and B
    - Confirmation of correct ring behaviour